

CS-317 OPERATING SYSTEMS

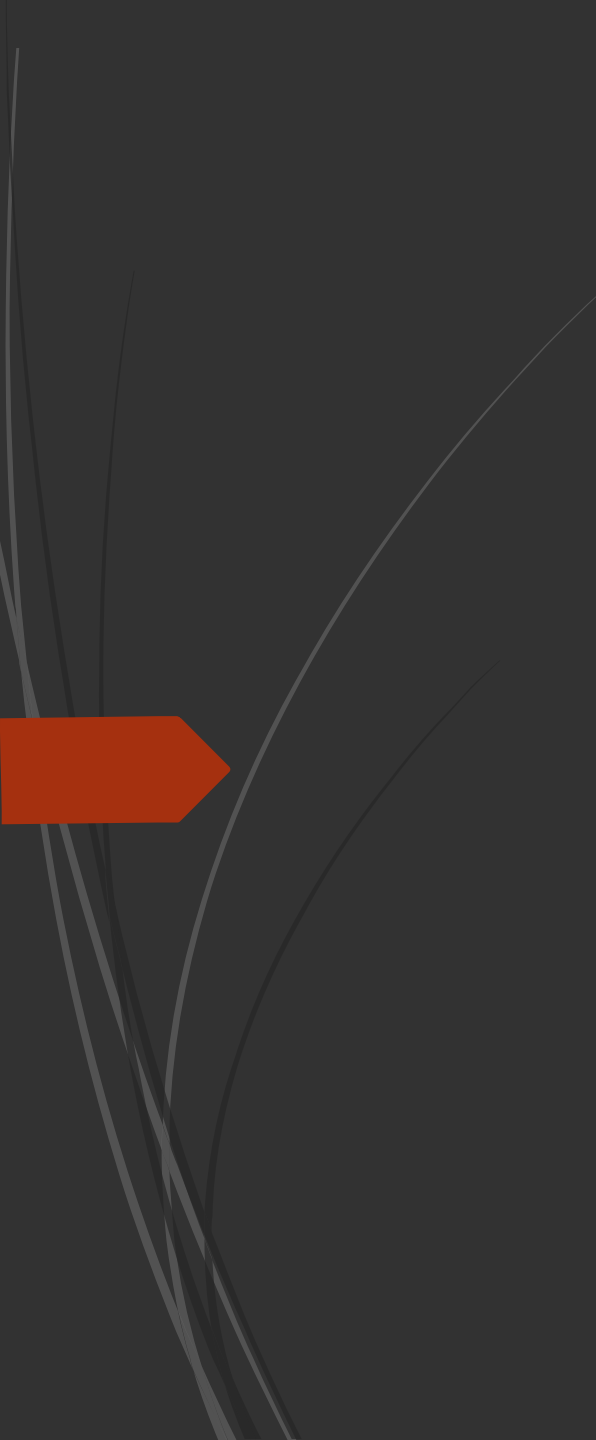
Lecture 21



File management

Book 1 Chapter 12





Overview

All about files



Files and file systems

Resource abstraction is a concept in computer science where complex or low-level resources are simplified and represented in a more manageable, high-level way so programmers (or systems) can use them without worrying about underlying details.

- From the user's point of view, one of the most important parts of an operating system is the file system.
- The file system provides the resource abstractions typically associated with secondary storage.
- The file system permits users to create data collections, called files.



Desirable file properties

- **Long-term existence:** Files are stored on disk or other secondary storage and do not disappear when a user logs off.
- **Sharable between processes:** Files have names and can have associated access permissions that permit controlled sharing.
- **Structure:** Depending on the file system, a file can have an internal structure that is convenient for applications. In addition, files can be organized into hierarchical or more complex structure to reflect the relationships among files.



Functions on files

- Any file system provides not only a means to store data organized as files, but a collection of functions that can be performed on files.
- Typical operations include the following:
 - **Create:** A new file is defined and positioned within the structure of files.
 - **Delete:** A file is removed from the file structure and destroyed.



Functions on files

- Any file system provides not only a means to store data organized as files, but a collection of functions that can be performed on files.
- Typical operations include the following:
 - **Open:** An existing file is declared to be "opened" by a process, allowing the process to perform functions on the file.
 - **Close:** The file is closed with respect to a process, so that the process no longer may perform functions on the file, until the process opens the file again.



Functions on files

- Any file system provides not only a means to store data organized as files, but a collection of functions that can be performed on files.
- Typical operations include the following:
 - **Read:** A process reads all or a portion of the data in a file.
 - **Write:** A process updates a file, either by adding new data that expands the size of the file or by changing the values of existing data items in the file.





File attributes

- Typically, a file system maintains a set of attributes associated with the file.
- These include:
 - Owner,
 - Creation time,
 - Time last modified,
 - Access privileges,and so on.



File structure

- Four terms are in common use when discussing files:
 - Field,
 - Record,
 - File,
 - Database.



Field columns

- A field is the basic element of data.
- An individual field contains a single value, such as an employee's last name, a date, or the value of a sensor reading.
- It is characterized by its length and data type (e.g., ASCII string, decimal).
- Depending on the file design, fields may be fixed length or variable length.



Variable-length fields

cnic = fixed length name = variable length

- In some cases, the field often consists of two or three subfields:
 - The actual value to be stored,
 - The name of the field, and
 - The length of the field.
- In other cases, the length of the field is indicated using special demarcation symbols between fields.



Record row

if there is variable length in field or record, there will be a length field

- A record is a collection of related fields that can be treated as a unit by some application program.
- Records may be of fixed length or variable length.
- A record will be of variable length if some of its fields are of variable length or if the number of fields may vary.
- In both kinds of variable-length records, the entire record usually includes a length field.
- In the case of variable number of fields, each field is usually accompanied by a field name.



File

- A file is a collection of similar records.
- The file is treated as a single entity by users and applications and may be referenced by name.
- Files have file names and may be created and deleted.
- Access control restrictions usually apply at the file level. That is, in a shared system, users and programs are granted or denied access to entire files.
- In some more sophisticated systems, such controls are enforced at the record or even the field level.
- Some file systems are structured only in terms of fields, not records. In that case, a file is a collection of fields.

Example : Simple list of values



Database

- A database is a collection of related data.
- The relationships that exist among elements of data in a database are explicit. *It means that in a database, the connections between data items are clearly defined and stored, rather than being implied or guessed*
- The database is designed for use by several different applications.
- A database may contain all the information related to an organization or project.
- The database itself consists of one or more types of files.
- Usually, there is a separate database management system that is independent of the operating system, although that system may make use of some file management programs.



Operations on files

Display all students in a class → Ali, Sara, Ahmed (everything together)

First, you retrieve record with ID = 1 (Ali)
Then Retrieve_Next gives you the next record → Sara (ID = 2)
Calling it again → Ahmed (ID = 3)
So, each time it returns the next record in sequence after the current one.

Retrieve next exaple

- **Retrieve_All:** Retrieve all the records of a file. This will be required for an application that must process all the information in the file at one time. This operation is often equated with the **term sequential processing**, because all the records are accessed in sequence.
- **Retrieve_One:** This requires the retrieval of just a single record. Interactive, **transaction-oriented applications** need this operation.
- **Retrieve_Next:** This requires the retrieval of the record that is "next" in some logical sequence to the most recently retrieved record. Some interactive applications, such as filling in forms, may require such an operation. A program that is performing a search may also use this operation.



Operations on files

Current record → Sara (ID = 2)
Retrieve_Prev → Ali (ID = 1)
If current = Ahmed → Retrieve_Prev → Sara

- **Retrieve_Previous:** It is similar to Retrieve_Next, but in this case the record that is "previous" to the currently accessed record is retrieved.
- **Insert_One:** Insert a new record into the file. It may be necessary that the new record fit into a particular position to preserve a sequencing of the file.
- **Delete_One:** Delete an existing record. Certain linkages or other data structures may need to be updated to preserve the sequencing of the file.



Operations on files

- **Update_One**: Retrieve a record, update one or more of its fields, and rewrite the updated record back into the file. Again, it may be necessary to preserve sequencing with this operation. If the length of the record has changed, the update operation is generally more difficult than if the length is preserved.
- **Retrieve_Few**: Retrieve several records. An application or user may wish to retrieve all records that satisfy a certain set of criteria.

`select * where =`





It should be noted that not all file systems exhibit the sort of structure discussed here.

On UNIX and UNIX-like systems, the basic file structure is just a stream of bytes.

A C program is stored as a file but does not have physical fields, records, and so on.



File management system

- A file management system is that set of system software that provides services to users and applications in the use of files.
- Typically, the only way that a user or application may access files is through the file management system.
- This relieves the user or programmer of the necessity of developing special-purpose software for each application and provides the system with a consistent, well-defined means of controlling files.



Objectives of file management system

- To meet the data management needs and requirements of the user, which include storage of data and the ability to perform operations on it.
- To guarantee, to the extent possible, that data in the file are valid.
- To optimize performance, both from the system's point of view in terms of overall throughput and from the user's point of view in terms of response time.
- To provide I/O support for a variety of storage device types.
- To minimize or eliminate the potential for lost or destroyed data.
- To provide a standardized set of I/O interface routines to user processes.
- To provide I/O support for multiple users, in the case of multiple-user systems.



User requirements regarding files

- Each user should be able to create, delete, read, write, and modify files.
- Each user may have controlled access to other users' files.
- Each user may control what types of accesses are allowed to the user's files.
- Each user should be able to restructure the user's files in a form appropriate to the problem.
- Each user should be able to move data between files.
- Each user should be able to back up and recover his or her files in case of damage.
- Each user should be able to access his or her files by name rather than by numeric identifier.



File system architecture

Device drivers

- At the lowest level, **device drivers** communicate directly with peripheral devices or their controllers or channels.
- A device driver is responsible for starting I/O operations on a device and processing the completion of an I/O request.
- For file operations, the typical devices controlled are disk and tape drives.
- Device drivers are usually considered to be part of the operating system.

It means that when a computer performs file operations (like open, read, write, save), it mainly interacts with storage hardware such as disks and tape drives, and these devices are managed by the system (via device drivers).

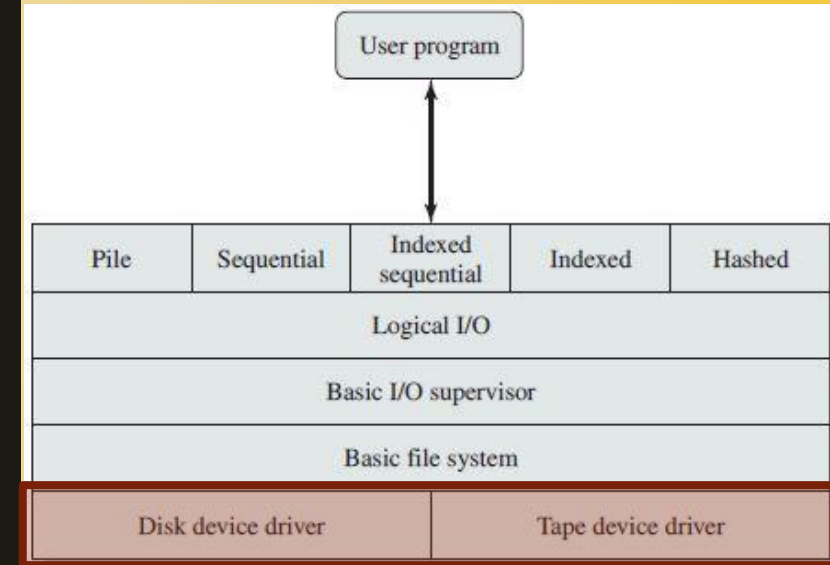


Figure 12.1 File System Software Architecture

Device drivers are special programs that allow the operating system to communicate with hardware devices.

You click "Print"
The OS sends a request
The printer driver converts it into instructions the printer understands



File system architecture

Basic file system

- The next level is referred to as the **basic file system**, or the **physical I/O** level.
- It deals with blocks of data that are exchanged with disk or tape systems.
- It is concerned with the placement of blocks on the secondary storage device and on the buffering of blocks in main memory.
- It does not understand the content of the data or the structure of the files involved.
- It is often considered part of the operating system.

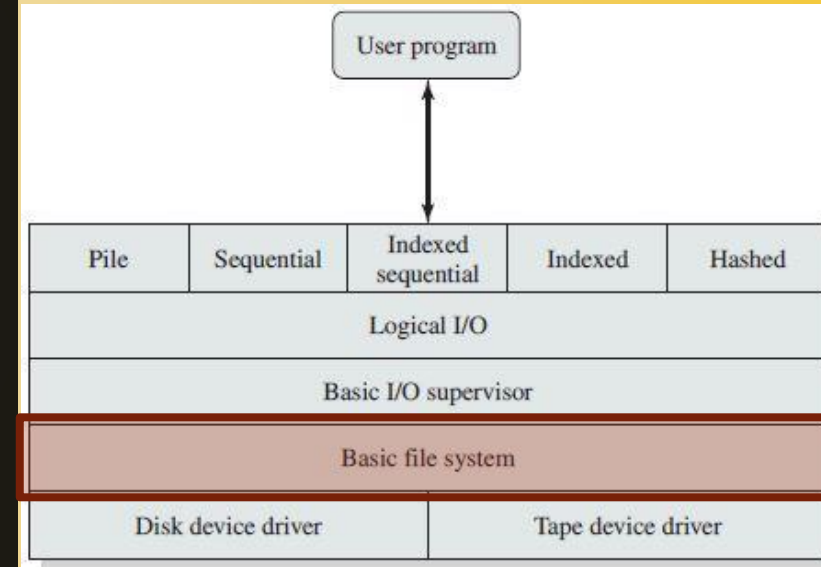


Figure 12.1 File System Software Architecture



File system architecture

Basic I/O supervisor

- The **basic I/O supervisor** is responsible for all file I/O initiation and termination.
- At this level, control structures are maintained that deal with device I/O, scheduling, and file status.
- The basic I/O supervisor selects the device on which file I/O is to be performed, based on the file selected.
- It is also concerned with scheduling disk and tape accesses to optimize performance.
- I/O buffers are assigned, and secondary memory is allocated at this level.
- It is part of the operating system.

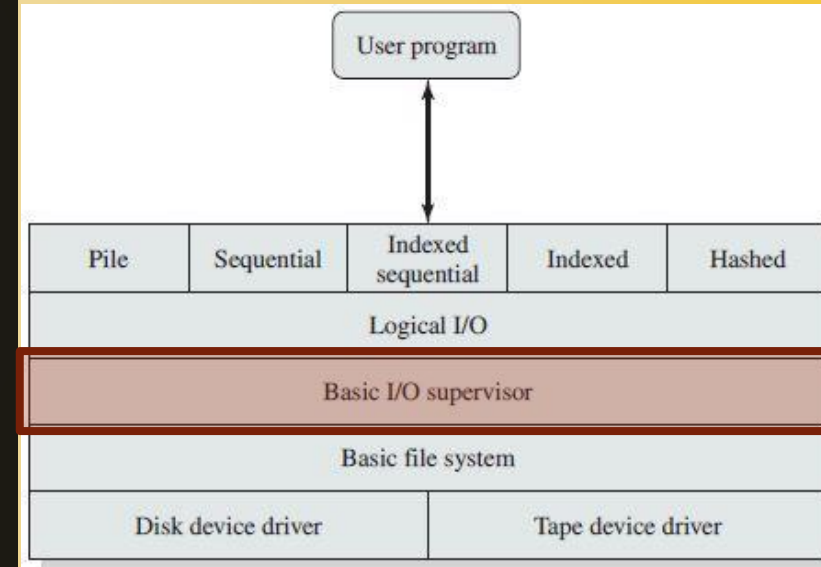


Figure 12.1 File System Software Architecture



File system architecture

Logical I/O

- Logical I/O enables users and applications to access records.
- Thus, whereas the basic file system deals with blocks of data, the logical I/O module deals with file records.
- Logical I/O provides a general-purpose record I/O capability and maintains basic data about files.

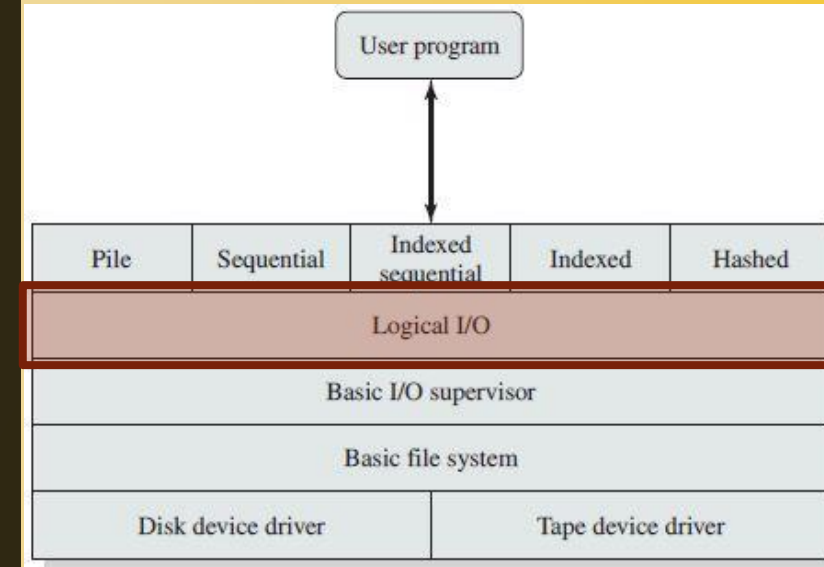


Figure 12.1 File System Software Architecture

Refers to operations performed by a program on a file
Involves reading or writing records without worrying about how data is physically stored or retrieved



File system architecture

Access method

- The level of the file system closest to the user is often termed the **access method**.
- It provides a standard interface between applications and the file systems and devices that hold the data.
- Different access methods reflect different file structures and different ways of accessing and processing the data.

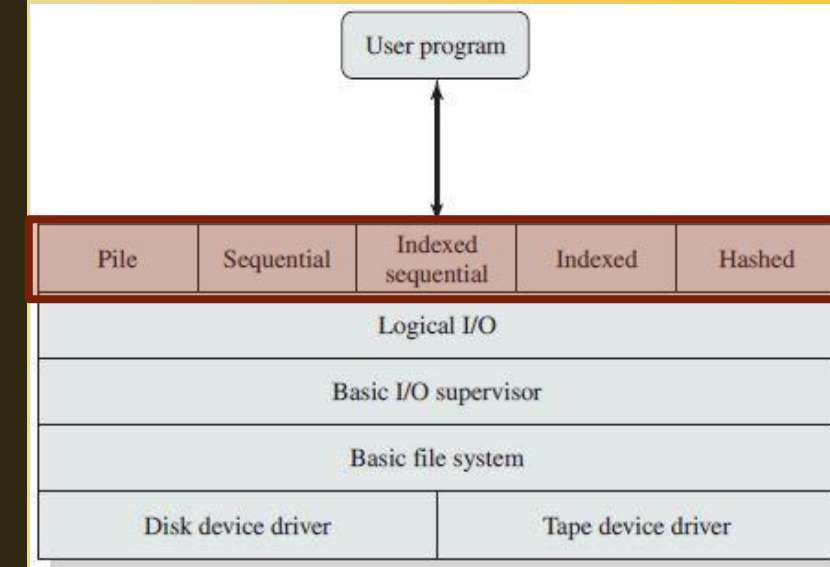


Figure 12.1 File System Software Architecture

Refers to the technique or strategy used by the OS to physically retrieve or store records on a device



File management functions

- Users and application programs interact with the file system by means of commands for creating, deleting and other operations on files.
- Before performing any operation, the file system must identify and locate the selected file.

File structure

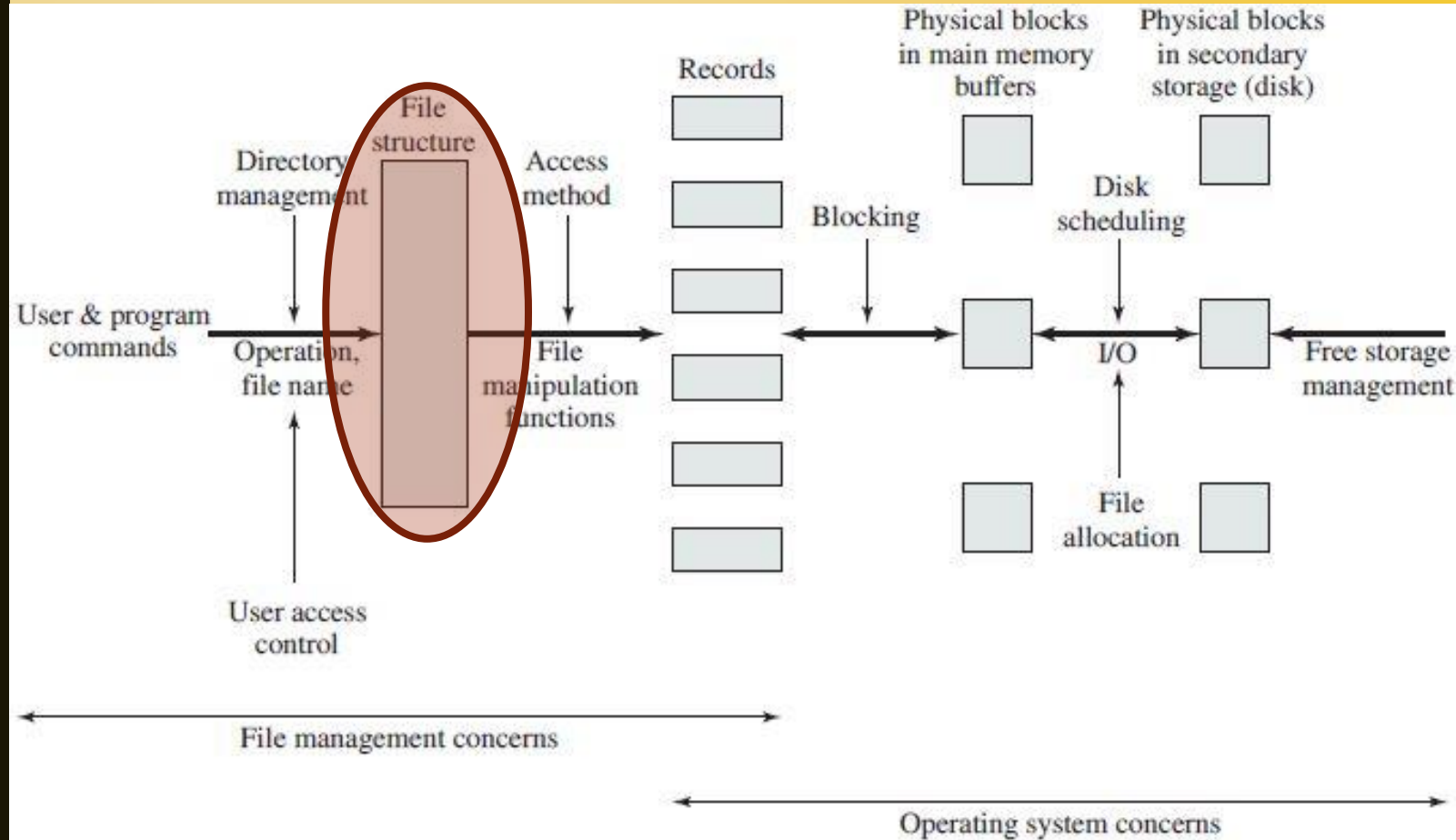


Figure 12.2 Elements of File Management



File management functions

- A directory describes the location of all files, plus their attributes.
- Most shared systems enforce user access control: only authorized users can access particular files in particular ways.

File structure

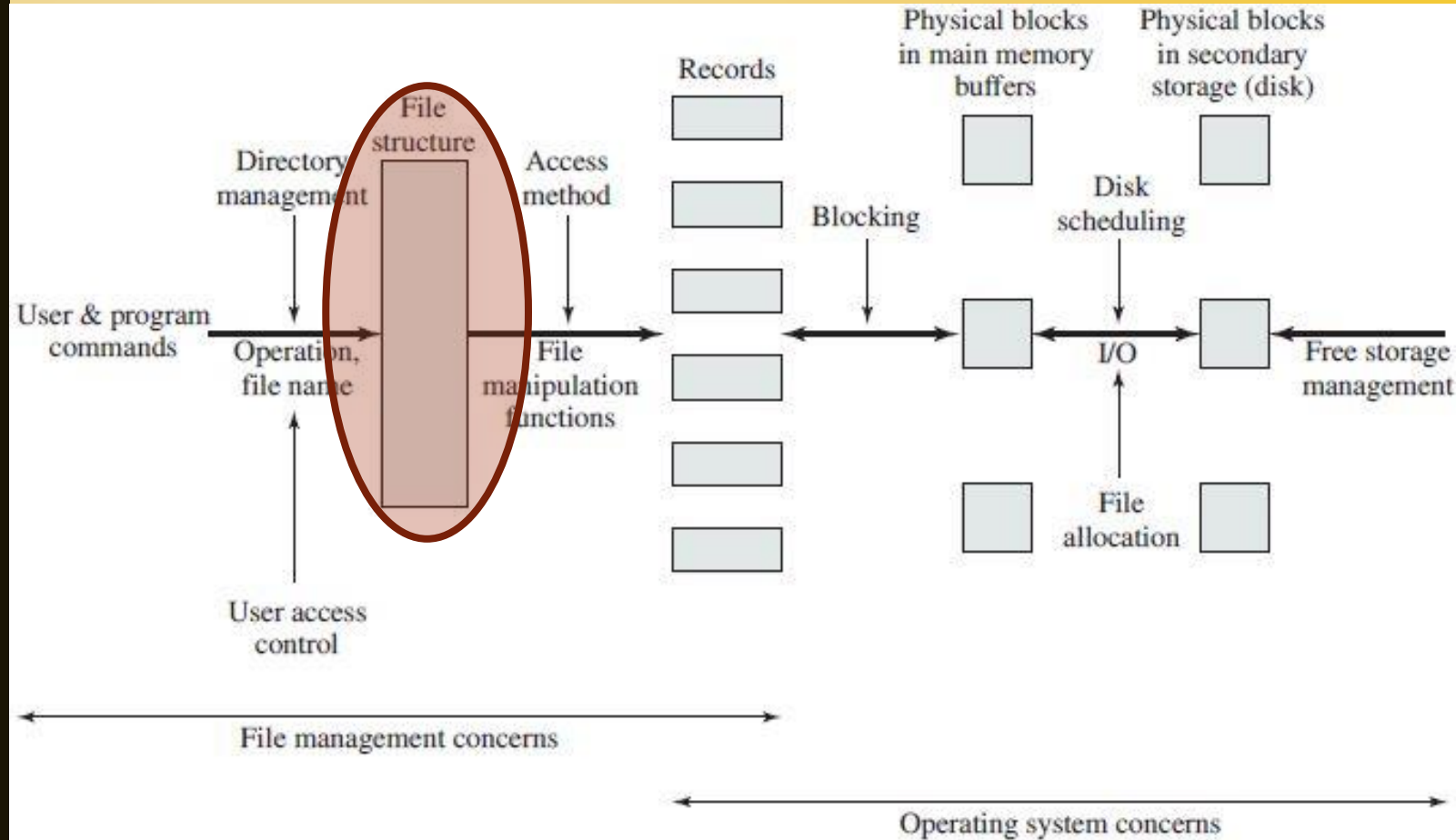


Figure 12.2 Elements of File Management



File management functions

- The basic operations on a file are performed at the record level.
- The user or application views the file as something that organizes the records.

Records

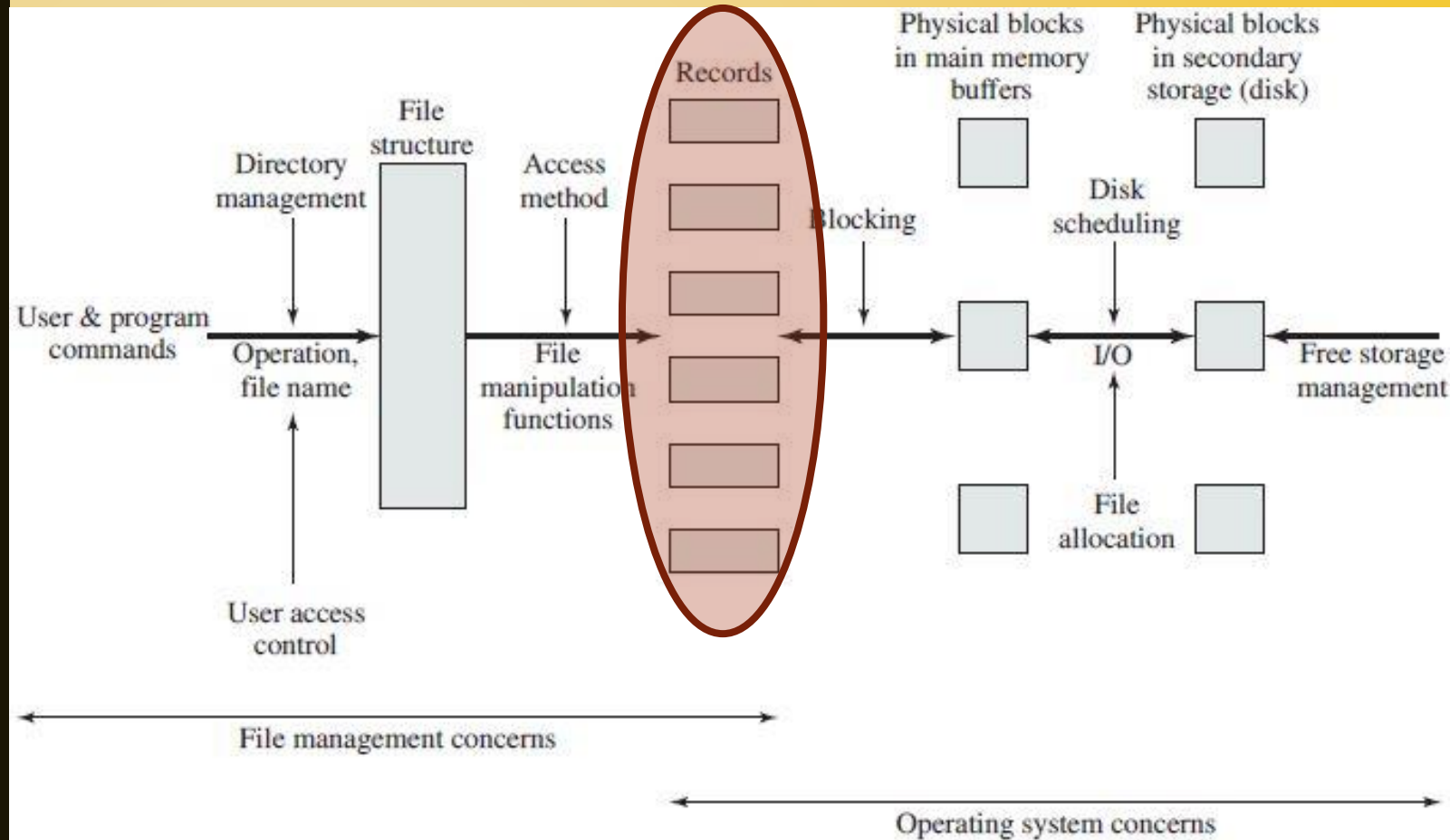


Figure 12.2 Elements of File Management



File management functions

- File I/O is done on a block basis.
- The records or fields of a file must be organized as a sequence of blocks for writing to the device.
- The blocks of a file must be organized as a sequence of records or fields after reading from the device.

Blocks in main memory

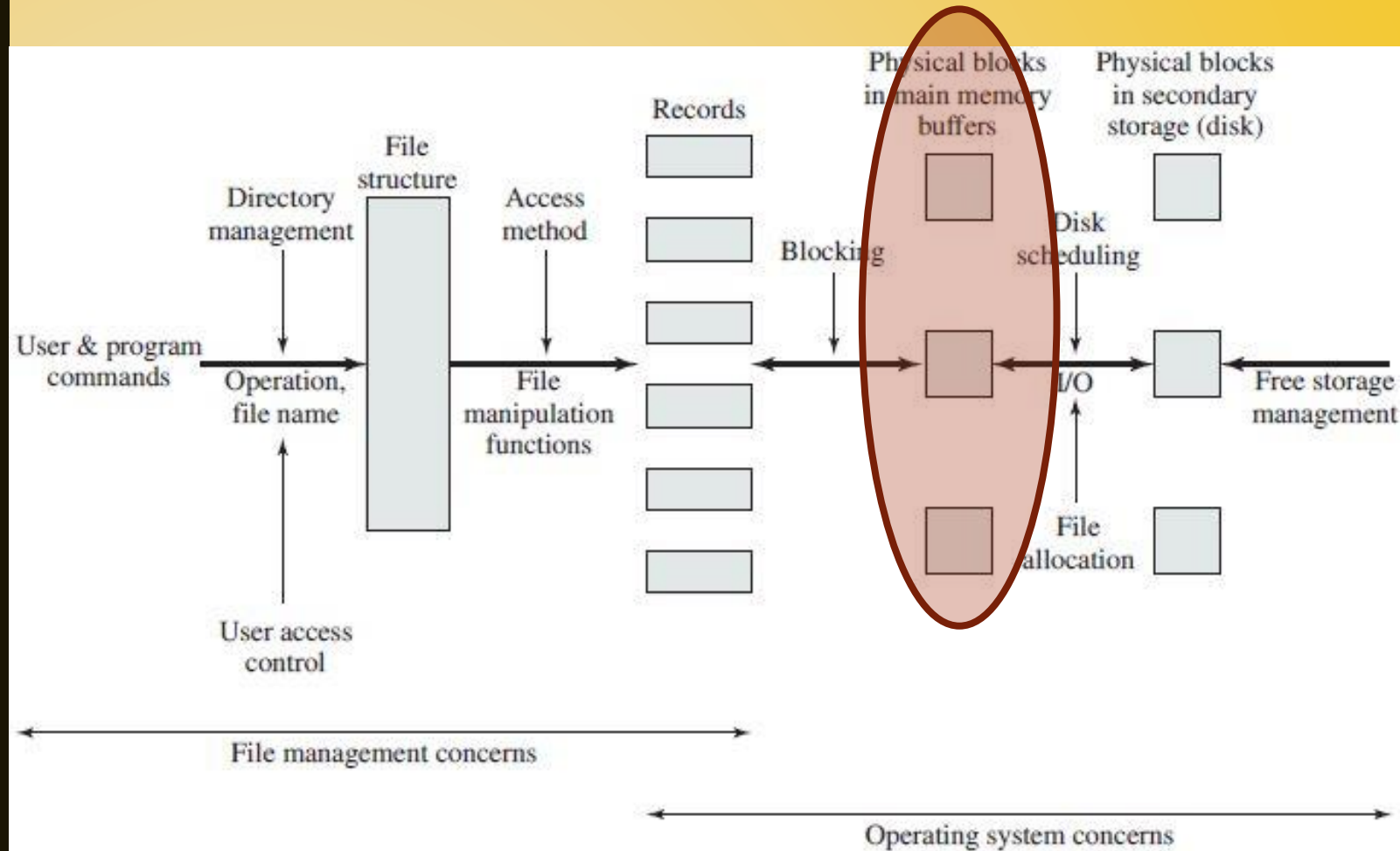


Figure 12.2 Elements of File Management



File management functions

- To support block I/O of files, we must allocate files to free blocks on secondary storage.
- We must manage free storage to know what blocks are available for new files and growth in existing files.
- Block I/O requests must be scheduled using a disk scheduling policy.

Blocks in secondary storage

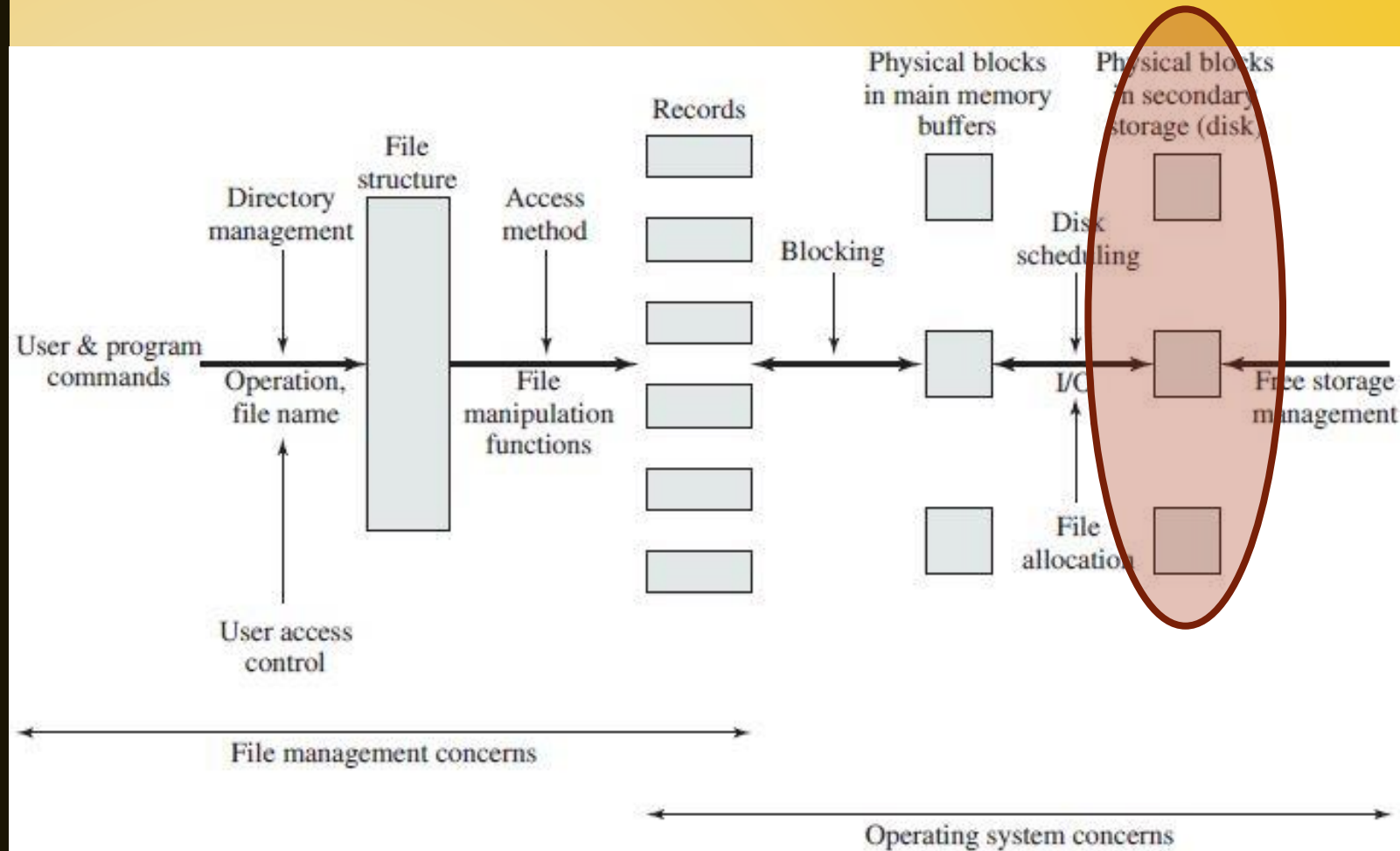


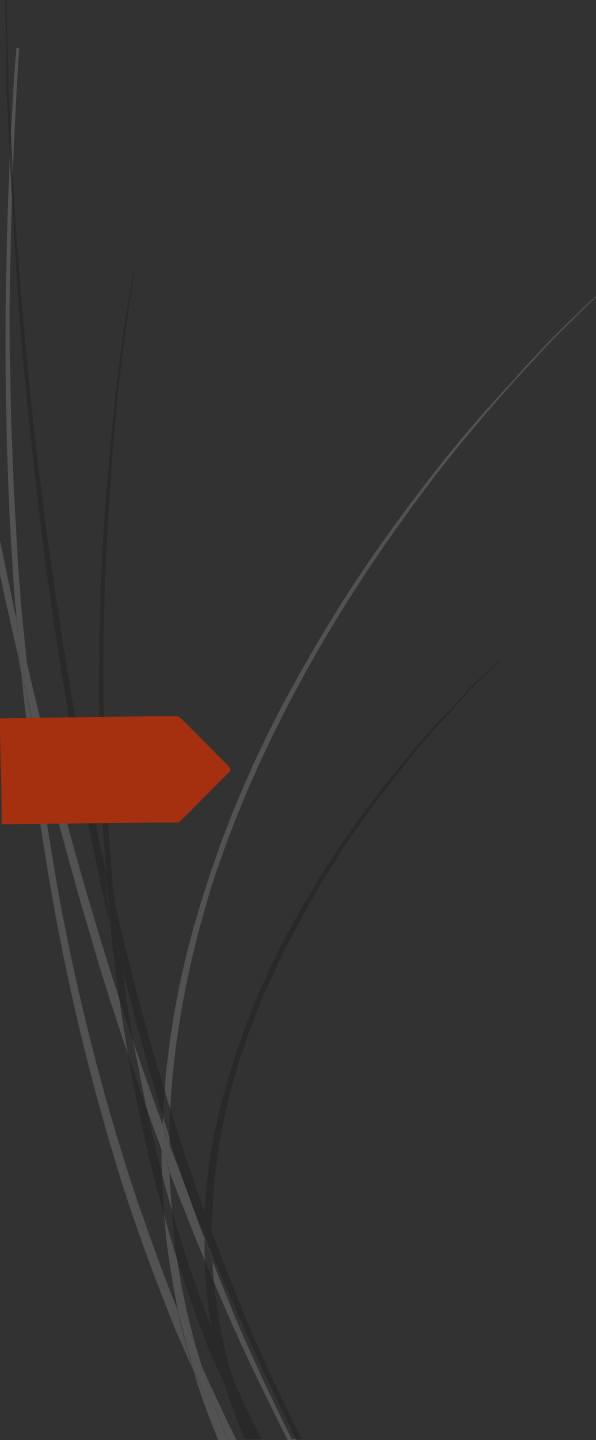
Figure 12.2 Elements of File Management



File system performance

- The optimization of a file system will depend on:
 - Disk scheduling policy,
 - File allocation strategy,
 - File structure, and
 - Access patterns.
- Developing an optimum file management system from the point of view of performance is an exceedingly complicated task.





File Organization and Access

Strategies and their pros and cons



File organization

- We use the term **file organization** to refer to the logical structuring of records.
- The physical organization of the file on secondary storage depends on:
 - The blocking strategy, and
 - The file allocation strategy.



Criteria to choose a file organization

- In choosing a file organization, several criteria are important:
 - Short access time,
 - Ease of update,
 - Economy of storage,
 - Simple maintenance,
 - Reliability.
- The relative priority of these criteria will depend on the applications that will use the file.
- These criteria may conflict.



Types of file organization

- We discuss five file organizations:
 - The pile,
 - The sequential file,
 - The indexed sequential file,
 - The indexed file, and
 - The direct, or hashed, file.
- Most structures used in actual systems either fall into one of these categories or can be implemented with a combination of these organizations.

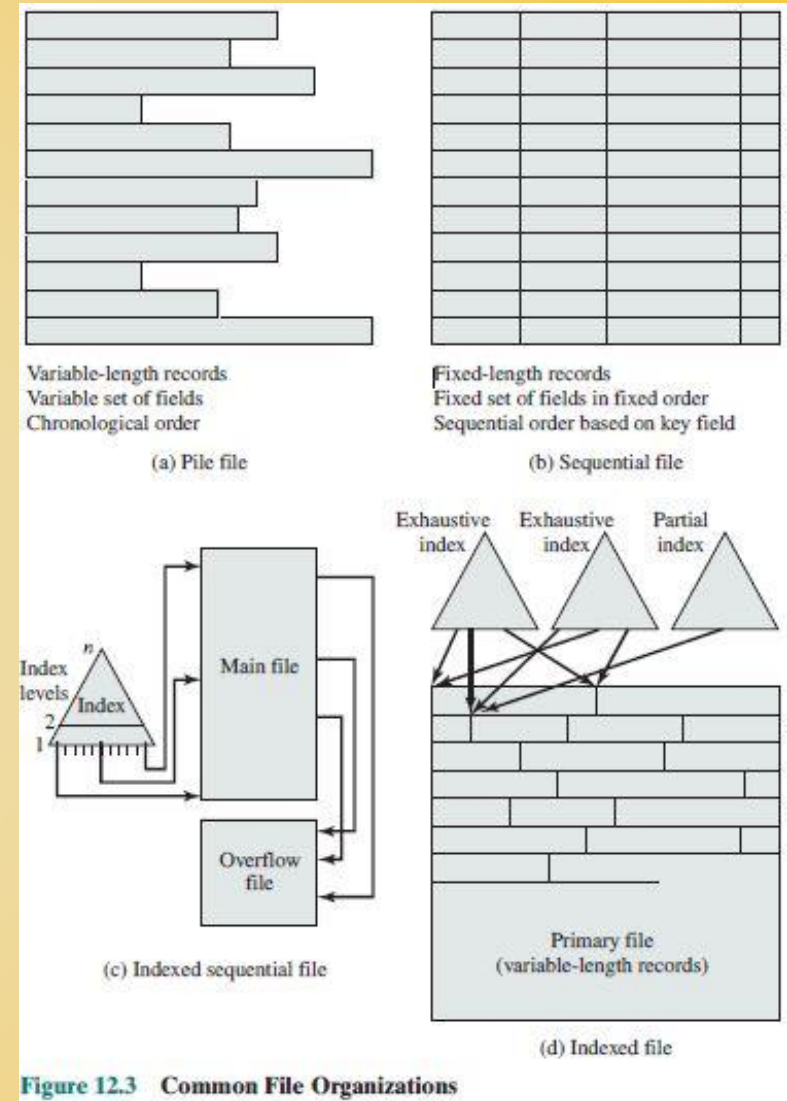
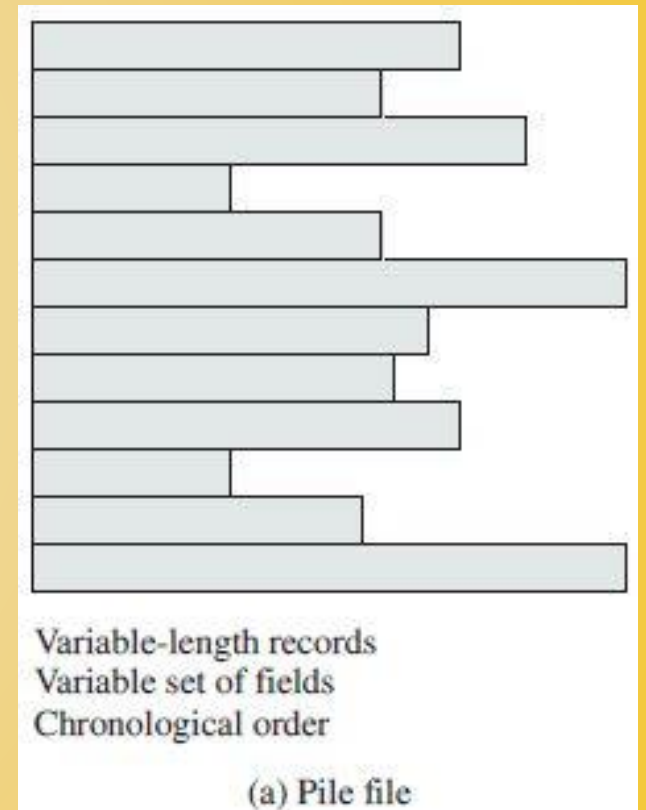


Figure 12.3 Common File Organizations



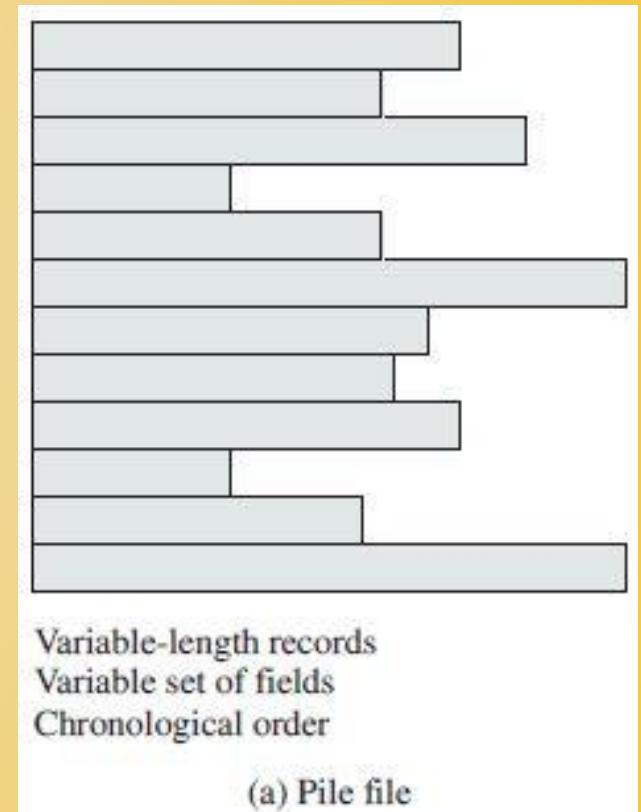
The pile

- The simplest file organization is the pile. Data are collected in the order in which they arrive. Each record consists of one burst of data.
- Records may have different fields, or similar fields in different orders.
- Each field should be self-describing, including a field name as well as a value.
- The length of each field must be:
 - Implicitly indicated by delimiters, or
 - Explicitly included as a subfield, or
 - Known as default for that field type.



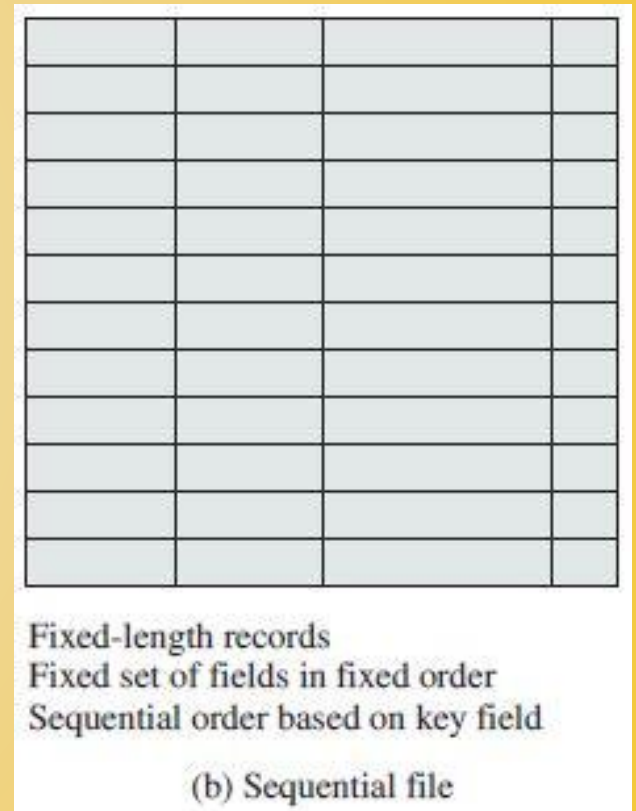
The pile

- A pile file **uses minimal space** and **is easy to update**.
- Pile files are made when **data are collected** and stored prior to processing or when data are not easy to organize.
- Due to lack of structure, record access is by **exhaustive search**. If we wish to find a record that contains a particular field with a particular value, we must examine each record in the pile until the desired record is found or the entire file has been searched. If we wish to find all records that contain a particular field with a particular value, the entire file must be searched.
- It is **unsuitable** for most applications.



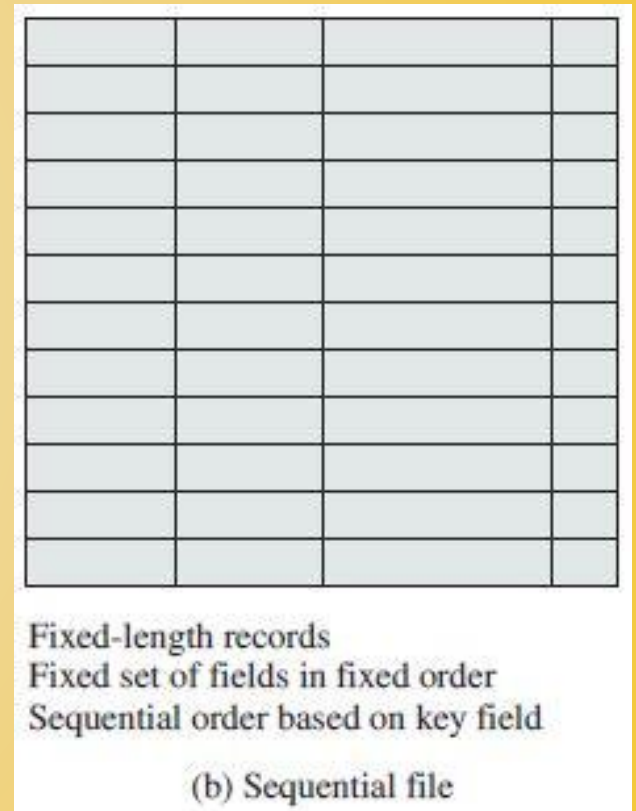
The sequential file

- The most common form of file structure is the sequential file. A **fixed format** is used for records. **All records are of the same length, consisting of the same number of fixed-length fields in a particular order.**
- Because the length and position of each field are known, **only the values of fields need to be stored; the field name and length for each field are attributes of the file structure.**
- The first field in each record, is referred to as the key field. The key field uniquely identifies the record.
- The records are stored in key sequence.



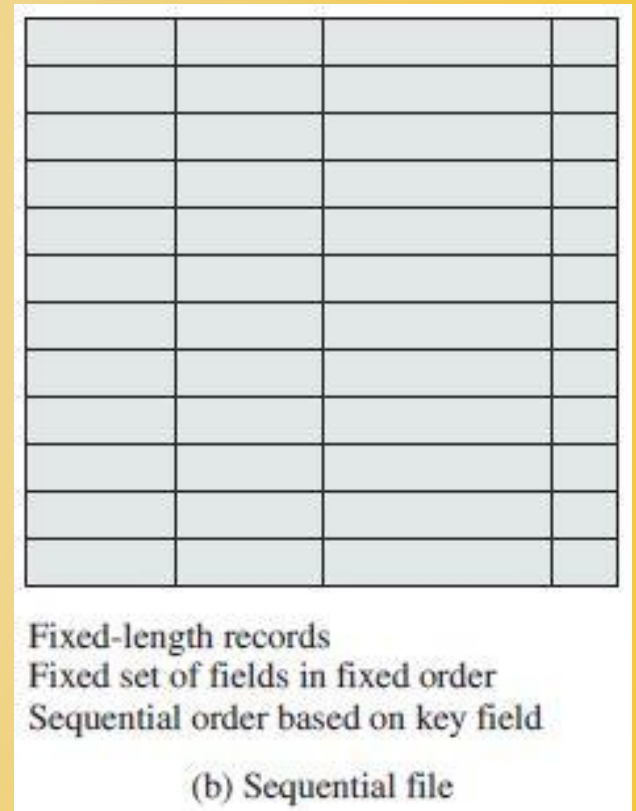
The sequential file

- A sequential file is stored in **sequential order** of the **records within blocks**. That is, the physical organization of the file on tape or disk directly matches the logical organization of the file.
- Sequential files are optimum for batch applications that involve the processing of all the records.
- To access a record, we need to do a **sequential search** of the file for a **key match**. If the entire file, or a large portion of the file can be brought into main memory at one time, more efficient search is possible.
- Every new record must be added in the proper place according to key order.



The sequential file

- The usual procedure is to place new records in a separate pile file, called a log file or transaction file. Periodically, a batch update is performed to merge the log file with the master file in correct key sequence.
- An **alternative** is to organize the sequential file physically as a **linked list**. One or more records are stored in each physical block. Each block on disk contains a pointer to the block containing the next record. The insertion of new records involves pointer manipulation but does not require that the records are in contiguous blocks. Thus, some added convenience is obtained at the cost of additional processing and overhead.





Pause the video!

Write your answer in a word file. Save it as <my_roll_no>_lecture_21. If your roll no is 500, the file will be named 500_lecture_21. Do not add <> signs to the filename.

Task# 1: Can you use the binary search algorithm on a pile or a sequential file? Why, or why not?

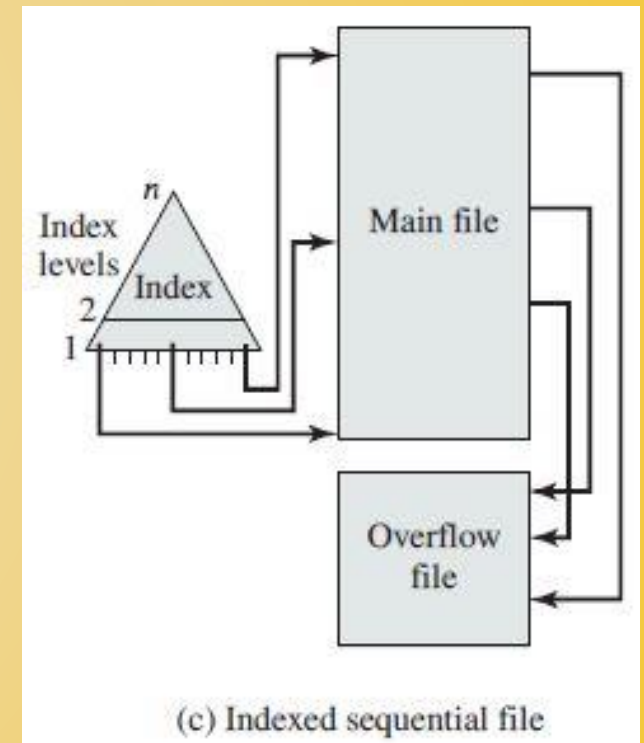
* **Pile file:** ☒ No, because records are unsorted.

* **Sequential file:** ☐ Yes, if records are stored in sorted key order, because binary search requires sorted data.



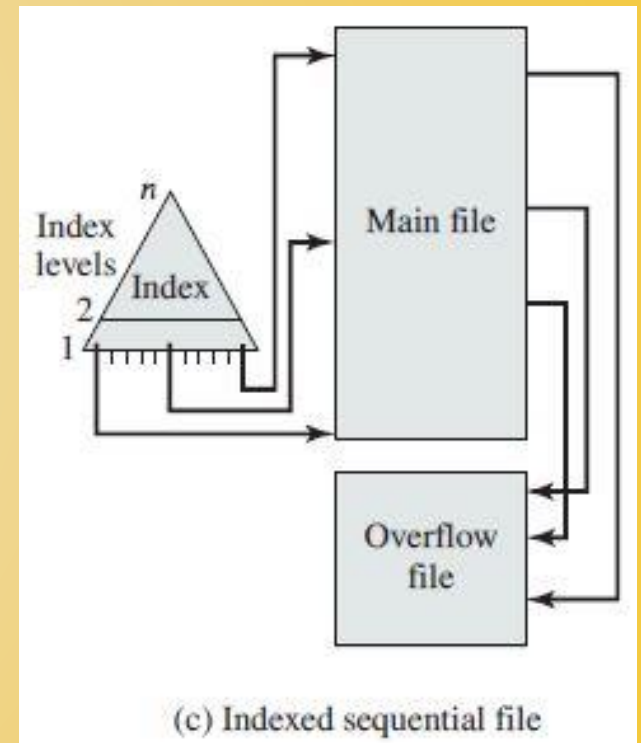
The indexed sequential file

- The indexed sequential file maintains the **key characteristic of the sequential file: records are organized in sequence based on a key field.**
- Two features are added: **an index to the file to support random access**, and an overflow file.
- The index provides a lookup capability to reach quickly the vicinity of a desired record.
- The overflow file is similar to the log file used with a sequential file but is integrated so that a record in the overflow file is located by following a pointer from its predecessor record.



The indexed sequential file

- In the simplest indexed sequential structure, a single level of indexing is used.
- The index in this case is a simple sequential file.
- Each record in the index file consists of two fields: a key field, which is the same as the key field in the main file, and a pointer into the main file.
- To find a specific field, the index is searched to find the highest key value that is equal to or precedes the desired key value.
- The search continues in the main file at the location indicated by the pointer.



In data structures and algorithms, we discuss that if we conduct a linear search in an array of N elements, the average number of comparisons required is $(N + 1) / 2$.

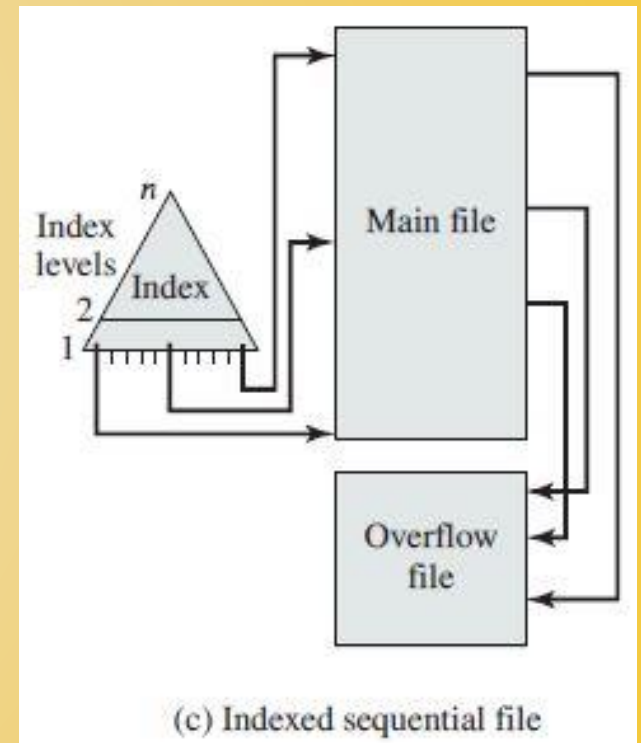
Using this, we state that during a sequential search of a file containing N records, we need to make $N / 2$ accesses in the average case.

A **sequential search** checks records one by one, so on average it examines **about half of the file**, i.e. **$N/2$ records (or comparisons)** before finding the target.



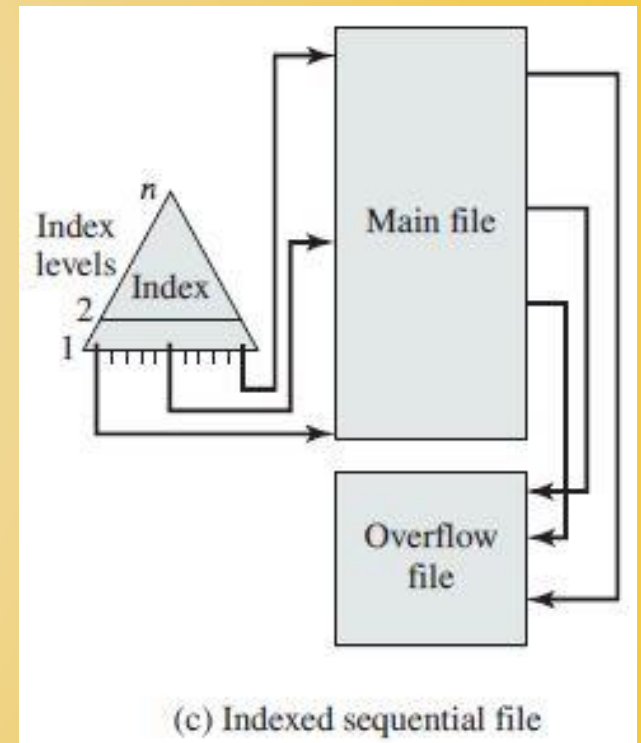
The indexed sequential file

- To see the effectiveness of this approach, consider a sequential file with 1,000,000 records.
- To search for a particular key value will require on average 500,000 record accesses.
- Now suppose that an index containing 1,000 entries is constructed, with the keys in the index evenly distributed over the main file. So each entry in the index points to a set of 1,000 ($= 1,000,000 / 1,000$) records.
- Now it will take on average 500 accesses to the index file followed by 500 accesses to a set (of 1,000 records) of the main file to find the record.
- The average number of accesses is reduced from 500,000 to 1,000.



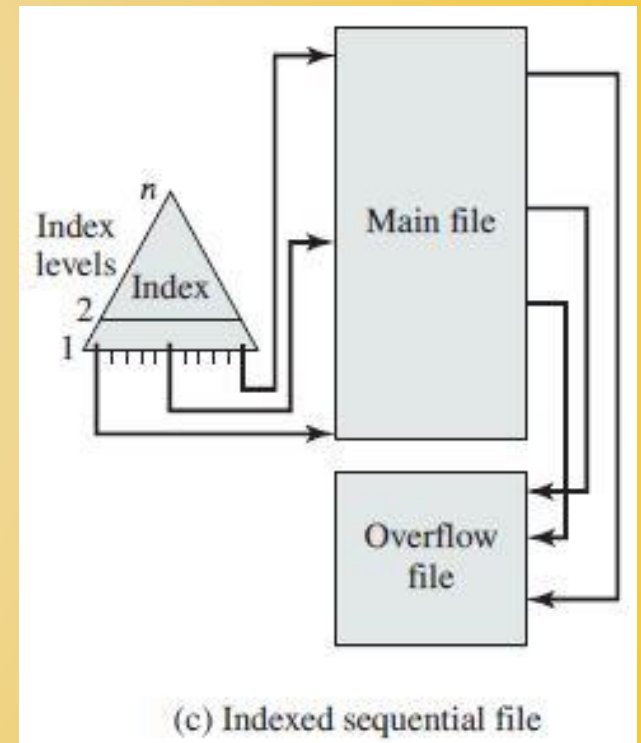
The indexed sequential file

- Additions to the file are handled in the following manner: each record in the main file contains an additional field not visible to the application, which is a pointer to the overflow file.
- When a new record is to be inserted into the file, it is added to the overflow file.
- The record in the main file that immediately precedes the new record in logical sequence is updated to contain a pointer to the new record in the overflow file.
- If the immediately preceding record is itself in the overflow file, then the pointer in that record is updated.



The indexed sequential file

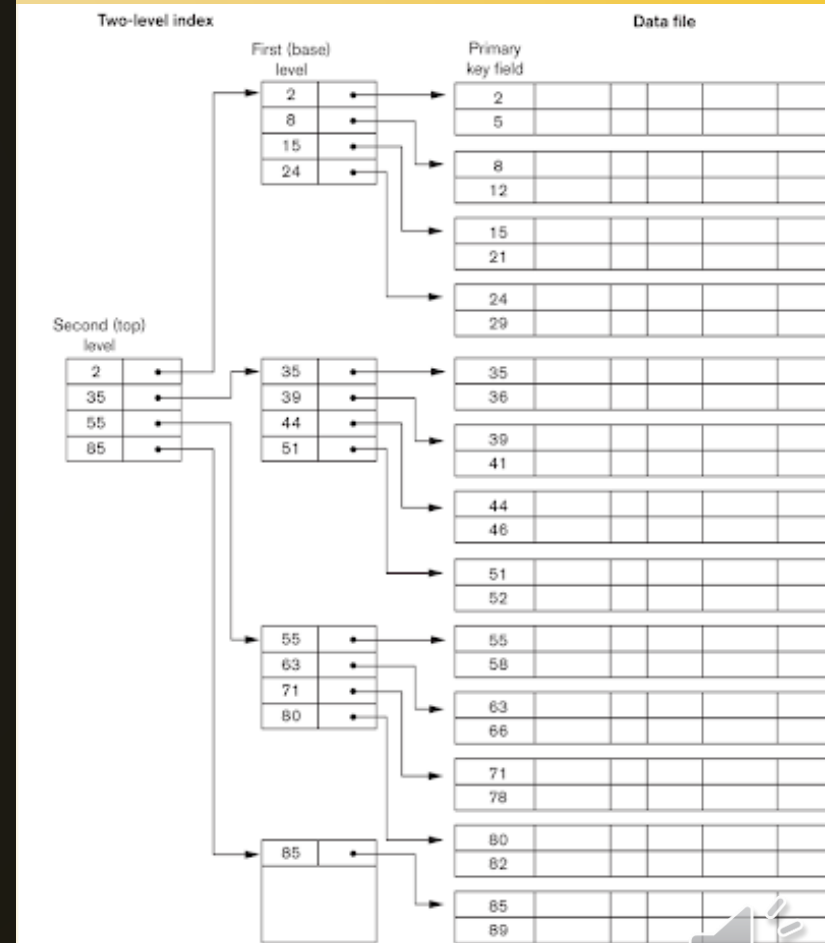
- The indexed sequential file is occasionally merged with the overflow file in batch mode.
- The indexed sequential file greatly reduces the time required to access a single record, without sacrificing the sequential nature of the file.
- To process the entire file sequentially, the records of the main file are processed in sequence until a pointer to the overflow file is found, then accessing continues in the overflow file until a null pointer is encountered, at which time accessing of the main file is resumed from where it was left off.
- A search is unsuccessful when we get to the end of the file.



The indexed sequential file

Records in file = 1,000,000
Entries in lower-level index = 10,000
Entries in higher-level index = 100
index = 100

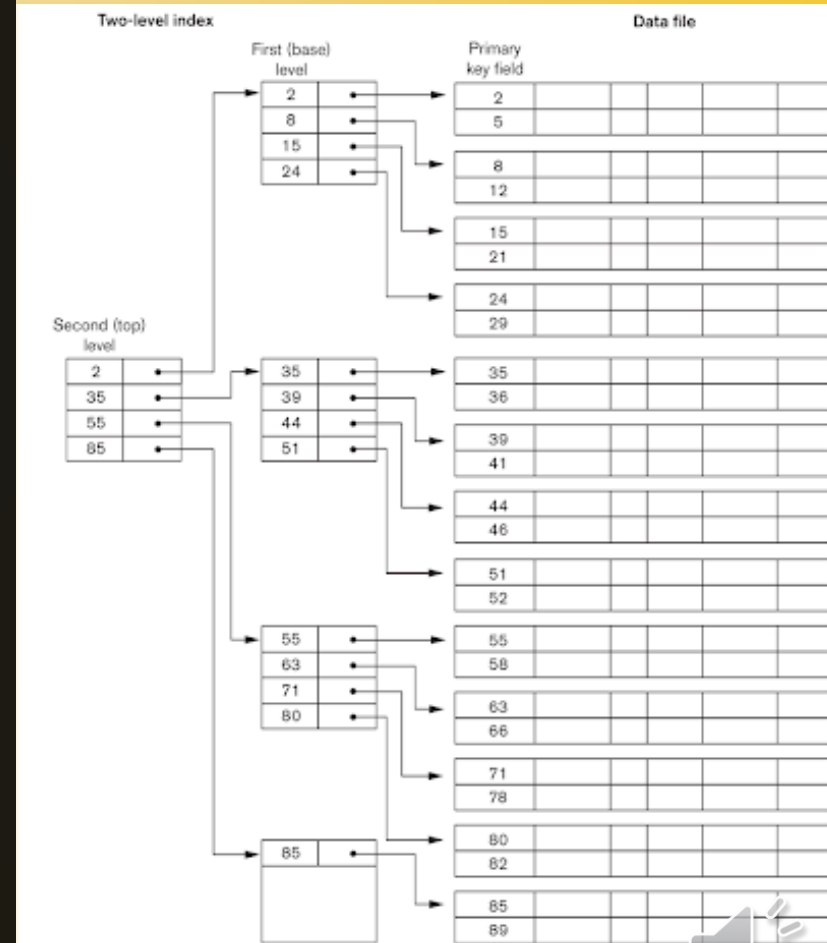
- To provide even greater efficiency in access, multiple levels of indexing can be used.
- The lowest level of index file is treated as a sequential file and a higher-level index file is created for that file.
- To understand, we assume a multi-level indexed file with 2 levels of indexes.
- Consider a file with 1,000,000 (1 million) records.
- A lower-level index with 10,000 entries is constructed. Each entry in this index points to a set of 100 ($= 1,000,000 / 10,000$) records.
- A higher-level index (into the lower-level index) of 100 entries is constructed. Each entry in this index points to a set of 100 ($= 10,000 / 100$) entries in the lower-level index.



The indexed sequential file

Records in file = 1,000,000
Entries in lower-level index = 10,000
Entries in higher-level index = 100
index = 100

- Each entry in the lower-level index points to a set of 100 ($= 1,000,000 / 10,000$) records.
- Each entry in the higher-level index points to a set of 100 ($= 10,000 / 100$) entries in the lower-level index.
- The search begins at the higher-level index (average 50 accesses) to find an entry point into the lower-level index.
- This set in the lower-level index (of 100 entries) is then searched (average 50 accesses) to find a pointer into the main file.
- This entry points to a set of 100 records in the main file which is then searched (average 50 accesses).
- The average number of accesses reduces from 500,000 to 1,000 to 150.



Pause the video!

Write your answer in a word file previously saved as
`<my_roll_no>_lecture_21`.

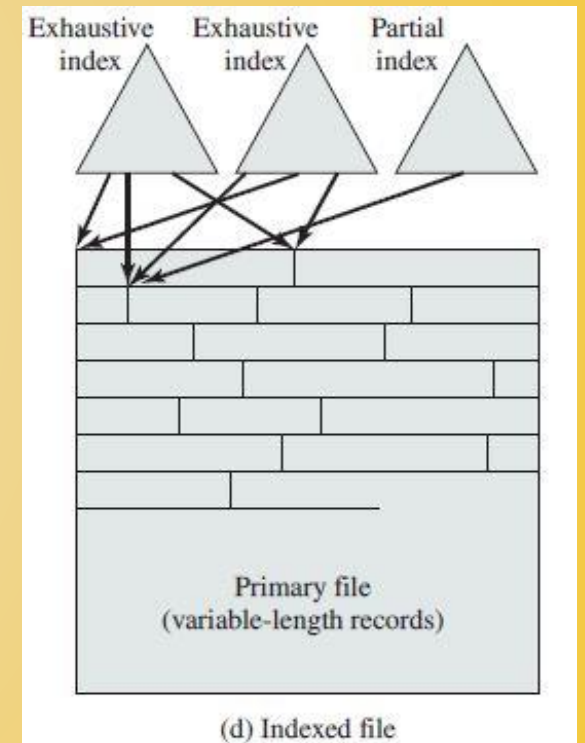
Task# 2: Consider a file with 2 million records. Find the average number of accesses for this file if:

- a. No index is used.
- b. An index of 5000 entries is used.
- c. Two indexes, a higher-level index of 50 entries and a lower-level index of 2000 entries, are used.



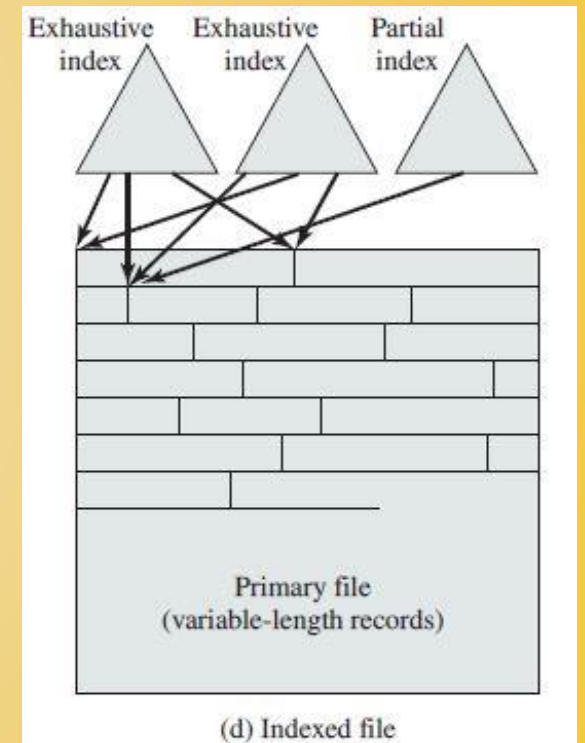
The indexed file

- In the general indexed file, the concept of sequentiality and a single key are abandoned. Records are accessed only through their indexes.
- There is now no restriction on the placement of records as long as a pointer in at least one index refers to that record.
- Two types of indexes are used.
 - An **exhaustive index** contains one entry for every record in the main file. The index itself is organized as a sequential file for ease of searching.
 - A **partial index** contains entries to records where the field of interest exists.



The indexed file

- Variable-length records can be employed.
- With variable-length records, some records will not contain all fields.
- When a new record is added to the main file, all of the index files must be updated.
- Indexed files are used mostly in applications where timeliness of information is critical and where data are rarely processed exhaustively.
- Examples are airline reservation systems and inventory control systems.



The direct, or hashed, file

- The direct, or hashed, file exploits the capability found on disks to access directly any block of a known address.
- As with sequential and indexed sequential files, a key field is required in each record.
- However, there is no concept of sequential ordering here.
- The direct file makes use of hashing on the key value.
- Direct files are often used where very rapid access is required, where fixed-length records are used, and where records are always accessed one at a time.
- Examples are directories, pricing tables, schedules, and name lists.





Pause the video!

Write your answer in a
word file previously saved
as
<my_roll_no>_lecture_21.

Task# 3: What is a hash
function with respect
to searching?



What did we learn?

- We discussed functions on files.
- We explored file structure.
- We discussed operations on files.
- We understood file management system and explored its objectives, architecture, functions and performance issues.
- We discussed file organization.
- We explored different types of file organization.



Things to do

